

Análise de Superescalabilidade no Algoritmo Quick Sort

Jozias Martini¹, Caroline Piazza², Marco Bernardeli³

¹Universidade Federal da Fronteira Sul (UFFS)
Chapecó - SC – Brazil

jozias.martini@gmail.com, karolinedq@gmail.com, marabdv@gmail.com

Abstract. *This work presents a superscalability analysis of the Quick Sort algorithm through the execution of RISC-V instructions using the gem5 simulator in a Linux environment. The objective of the study is to evaluate the impact of instruction-level parallelism on the algorithm's performance under different simulated processor configurations. To achieve this, a Quick Sort implementation was developed in RISC-V and executed within the gem5 simulator, enabling the collection of metrics related to clock cycles, executed instructions, and overall system performance. The results demonstrate how superscalar architectures can reduce execution time by dispatching multiple instructions simultaneously, highlighting both the benefits and limitations of superscalability in applications with different instruction dependency patterns.*

Resumo. *Este trabalho apresenta uma análise de superescalabilidade aplicada ao algoritmo Quick Sort por meio da execução de instruções da arquitetura RISC-V utilizando o simulador gem5 em ambiente Linux. O objetivo do estudo é avaliar o impacto da exploração do paralelismo em nível de instrução no desempenho do algoritmo, considerando diferentes configurações de processadores simulados. Para isso, foi implementada uma versão do Quick Sort em RISC-V e executada no simulador gem5, permitindo a coleta de métricas relacionadas ao número de ciclos, instruções executadas e desempenho geral do sistema. Os resultados obtidos demonstram como arquiteturas superescalares podem reduzir o tempo de execução ao despachar múltiplas instruções simultaneamente, evidenciando os benefícios e limitações da superescalabilidade em aplicações com diferentes padrões de dependência entre instruções.*

1. Introdução

A crescente demanda por desempenho computacional tem impulsionado o desenvolvimento de arquiteturas de processadores capazes de executar instruções de forma cada vez mais eficiente. Entre as técnicas utilizadas para aumentar o desempenho dos sistemas computacionais destaca-se a superescalabilidade, que permite a execução simultânea de múltiplas instruções em um mesmo ciclo de clock. A avaliação dessas arquiteturas é fundamental para compreender seus benefícios e limitações em aplicações reais, especialmente em algoritmos amplamente utilizados na computação.

O Quick Sort é um dos algoritmos de ordenação mais conhecidos e eficientes da ciência da computação. Desenvolvido por Tony Hoare em 1959, o algoritmo utiliza a estratégia de divisão e conquista para ordenar elementos de uma lista. Seu funcionamento consiste na escolha de um elemento pivô e na divisão do conjunto de dados em subconjuntos menores, que são ordenados recursivamente. Em média, o Quick Sort apresenta

complexidade temporal de $O(n \log n)$, tornando-se uma das soluções mais utilizadas para ordenação de dados em aplicações práticas.

Para a execução do algoritmo neste estudo, foi utilizada a arquitetura RISC-V, um conjunto de instruções (Instruction Set Architecture, ou ISA) aberto e de livre utilização. Diferentemente de arquiteturas proprietárias, o RISC-V oferece flexibilidade para pesquisa, ensino e desenvolvimento de processadores personalizados. Sua filosofia é baseada nos princípios da computação com conjunto reduzido de instruções (Reduced Instruction Set Computer, ou RISC), proporcionando simplicidade, modularidade e eficiência na implementação de hardware e software.

A análise do comportamento do Quick Sort foi realizada por meio do simulador gem5, uma plataforma amplamente utilizada na pesquisa acadêmica e industrial para modelagem e avaliação de arquiteturas computacionais. O gem5 permite simular diferentes configurações de processadores, memórias e sistemas computacionais, possibilitando a coleta de métricas detalhadas de desempenho, como número de ciclos, instruções executadas e taxa de utilização dos recursos do processador. Dessa forma, torna-se possível investigar o impacto de diferentes níveis de superescalabilidade sobre a execução de programas.

Neste contexto, o presente trabalho tem como objetivo analisar o desempenho do algoritmo Quick Sort implementado em RISC-V e executado no simulador gem5 em ambiente Linux, avaliando os efeitos da superescalabilidade sobre o tempo de execução e a eficiência do processamento. Os resultados obtidos contribuem para a compreensão da relação entre algoritmos de ordenação e arquiteturas modernas de processadores, evidenciando como o paralelismo em nível de instrução pode influenciar o desempenho computacional.

2. Contextualização

Para a realização deste trabalho foi utilizado um ambiente Linux Ubuntu configurado para o desenvolvimento e execução de programas na arquitetura RISC-V utilizando o simulador gem5. O objetivo foi analisar o comportamento do algoritmo Quick Sort em uma arquitetura simulada, possibilitando a coleta de métricas relacionadas ao desempenho do processador e à execução das instruções.

Inicialmente, o algoritmo Quick Sort foi implementado em linguagem Assembly para a arquitetura RISC-V. Como o código foi desenvolvido seguindo a sintaxe utilizada no simulador Venus, algumas adaptações foram necessárias para torná-lo compatível com o ambiente do gem5. Entre as modificações realizadas destacam-se a adequação da sintaxe das instruções, a implementação correta da função principal (*main*), o gerenciamento da pilha para preservação do registrador de retorno, a substituição de endereços de memória estáticos por seções de dados apropriadas e a adaptação das chamadas de sistema para utilização das bibliotecas fornecidas pela toolchain GCC. Essas alterações foram necessárias para permitir a execução correta do programa no sistema operacional emulado pelo gem5.

Após a adaptação do código Assembly, foi utilizada a ferramenta *riscv64-linux-gnu-gcc* para compilar o arquivo fonte (.s) e gerar um executável compatível com a arquitetura RISC-V. O processo de compilação produziu um arquivo binário no formato *.riscv*, que posteriormente foi utilizado durante a simulação.

Para a execução dos experimentos, foi utilizado um script de configuração baseado no modelo *Out-of-Order* do gem5, responsável por instanciar a arquitetura simulada e carregar o programa RISC-V compilado. A utilização desse modelo permitiu explorar características relacionadas à superescalabilidade, uma vez que processadores *Out-of-Order* são capazes de reorganizar a execução das instruções para aumentar o paralelismo e melhorar o aproveitamento dos recursos internos do processador.

Durante a simulação, o gem5 gerou informações detalhadas sobre a execução do programa. Parte desses resultados foi apresentada diretamente no terminal durante a execução da simulação, enquanto métricas mais completas foram registradas no arquivo *stats.txt*. Esse arquivo contém informações como número de ciclos de clock, quantidade de instruções executadas, IPC (*Instructions Per Cycle*), acessos à memória e outras estatísticas relevantes para a análise de desempenho.

A partir dessas métricas, foi possível avaliar o comportamento do algoritmo Quick Sort na arquitetura simulada, observando como os mecanismos de paralelismo em nível de instrução influenciam o desempenho da aplicação. Dessa forma, o ambiente composto por Ubuntu, toolchain RISC-V e simulador gem5 forneceu uma plataforma adequada para investigar os efeitos da superescalabilidade sobre a execução do algoritmo estudado.

3. Metodologia

A metodologia adotada neste trabalho consistiu na avaliação do impacto da superescalabilidade sobre a execução do algoritmo Quick Sort implementado em Assembly RISC-V. Os experimentos foram realizados em ambiente Linux Ubuntu utilizando o simulador gem5, ferramenta amplamente empregada na pesquisa de arquiteturas computacionais para análise detalhada de desempenho.

Inicialmente, o algoritmo Quick Sort foi implementado em linguagem Assembly para a arquitetura RISC-V. O programa realiza a ordenação de um vetor contendo dez elementos inteiros armazenados inicialmente na seção de dados do programa. Durante a execução, os dados são copiados para uma área de memória alocada dinamicamente e, em seguida, o algoritmo de ordenação é executado sobre esse vetor.

Após a implementação, o código-fonte Assembly foi compilado utilizando a ferramenta *riscv64-linux-gnu-gcc*, gerando um executável estático compatível com a arquitetura RISC-V. O comando utilizado foi:

```
riscv64-linux-gnu-gcc -static quick-sort.s -o quick-sort.riscv
```

O binário gerado foi executado no simulador gem5 utilizando um modelo de processador fora de ordem (*Out-of-Order*), baseado na classe DerivO3CPU. O tema estudado neste trabalho foi a superescalaridade, que consiste na capacidade do processador de despachar e executar múltiplas instruções em um mesmo ciclo de clock.

Para avaliar o impacto dessa característica, foram configurados dois cenários distintos de execução:

- Processador escalar: largura dos estágios configurada com valor igual a 1.
- Processador superescalar: largura dos estágios configurada com valor igual a 4.

Conforme proposto no enunciado da atividade, foram alterados simultaneamente os parâmetros *fetchWidth*, *decodeWidth*, *issueWidth* e *commitWidth*, permitindo comparar diretamente uma arquitetura escalar tradicional com uma arquitetura superescalar.

Após cada simulação, o gem5 gerou o arquivo *stats.txt*, contendo estatísticas detalhadas da execução. As principais métricas analisadas foram:

- *simTicks*: tempo total de simulação;
- *system.cpu.ipc*: instruções executadas por ciclo (IPC);
- Número total de instruções executadas;
- Quantidade total de ciclos de clock utilizados.

As métricas extraídas foram organizadas em tabelas e gráficos comparativos, permitindo observar como o aumento da largura dos estágios influencia o desempenho do algoritmo Quick Sort. A comparação entre os cenários possibilita identificar os ganhos obtidos pela exploração do paralelismo em nível de instrução e avaliar a eficiência da arquitetura superescalar para esse tipo de aplicação.

4. Resultados

Os resultados obtidos demonstram que a transição para uma arquitetura superescalar proporcionou ganhos de desempenho por meio da exploração do paralelismo em nível de instrução. Embora o aumento da largura do pipeline de uma para quatro vias permita, teoricamente, a conclusão de até quatro instruções por ciclo de clock, o IPC médio observado foi de apenas 0,71. Esse resultado indica que os mecanismos de paralelismo disponíveis na arquitetura foram significativamente limitados pelas características do algoritmo *Quick Sort*.

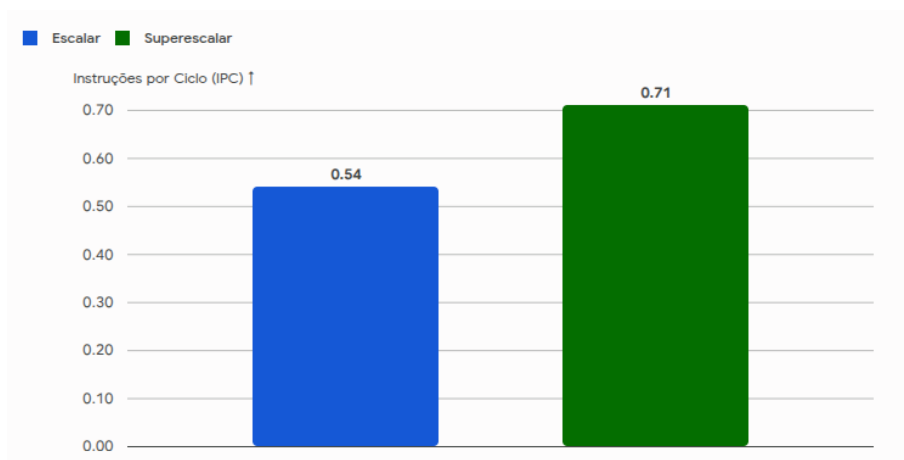


Figura 1. Instruções por ciclo.

O desempenho alcançado ficou aquém do potencial teórico da arquitetura devido às dependências inerentes ao algoritmo *Quick Sort*. Grande parte das instruções apresenta dependências de dados, exigindo que operações subsequentes aguardem a conclusão de acessos à memória e comparações anteriores, o que reduz a disponibilidade de instruções independentes para execução simultânea. Além disso, o algoritmo possui elevada frequência de desvios condicionais associados às comparações e particionamentos do vetor. Como arquiteturas superescalares dependem fortemente de mecanismos de predição de desvios para manter o pipeline alimentado, erros de predição resultam na invalidação das instruções especulativamente executadas, ocasionando desperdício de ciclos de processamento e redução da eficiência global. Consequentemente, mesmo dispondo de quatro vias de execução, o processador não consegue explorá-las plenamente, limitando o ganho efetivo de desempenho observado.

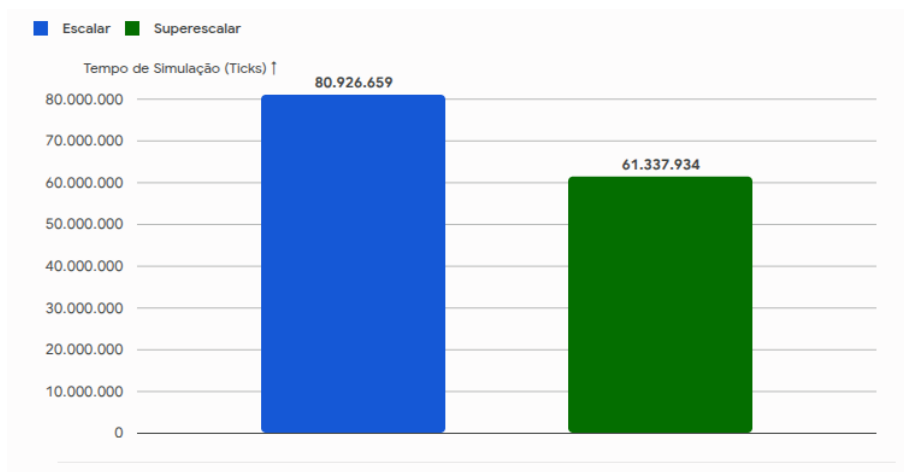


Figura 2. Tempo de simulação.

5. Conclusão

Este trabalho teve como objetivo avaliar o impacto da superescalaridade na execução do algoritmo *Quick Sort* por meio de simulações realizadas no gem5, utilizando o modelo de processador *Out-of-Order* baseado na arquitetura *RISC-V*. A comparação entre uma configuração escalar, com largura de emissão igual a uma instrução por ciclo, e uma configuração superescalar, com largura de quatro instruções por ciclo, permitiu analisar os efeitos da exploração do paralelismo em nível de instrução sobre o desempenho da aplicação.

Os resultados obtidos demonstraram que o aumento da largura da CPU proporcionou uma redução aproximada de 24% no tempo de execução, além de elevar o IPC médio de 0,54 para 0,71. Entretanto, observou-se que o ganho de desempenho não foi proporcional ao aumento dos recursos de *hardware* disponíveis. A utilização das vias adicionais do processador superescalar foi limitada pelas características intrínsecas do algoritmo *Quick Sort*, especialmente pela presença de dependências de dados e pela elevada incidência de desvios condicionais, fatores que reduzem a disponibilidade de instruções independentes para execução paralela e aumentam o impacto de erros de predição de desvios.

Dessa forma, os resultados evidenciam que o desempenho de arquiteturas superescalares depende não apenas da ampliação dos recursos de *hardware*, mas também da capacidade do *software* em fornecer paralelismo explorável. Embora o *hardware* disponibilize recursos para a exploração do paralelismo, é o *software* que, em última instância, determina o ritmo e a eficiência da execução.